

UNIVERSIDADE DO VALE DO ITAJAÍ

ESCOLA POLITÉCNICA

GUILHERME MOHR DOS SANTOS

ARTHUR RODRIGUES HAWRELIUK

DNS

Descoberta de serviços (cloud native)

Desenvolvimento do Trabalho M2 — 21/10/2026

ITAJAÍ

2026

1. Identificação e objetivo
2. Pesquisa e questionário
3. Ambiente de testes
4. Experimento E1 — Subida e topologia
    4.1. Previsão
    4.2. Execução e evidências
    4.3. Análise
5. Próximas etapas

1. Identificação e objetivo

Disciplina: Sistemas Distribuídos. **Professor:** Ramices dos Santos Silva.
Equipe: Guilherme Mohr dos Santos e Arthur Rodrigues Hawreliuk.
Tema: T1 — Sistema de nomes distribuído (DNS), com BIND 9 como sistema de referência.
Ênfase: D — Descoberta de serviços (cloud native), com CoreDNS no experimento E9.

Este documento registra o que a equipe desenvolveu na aula de 21/10/2026: a pesquisa e o rascunho do questionário, a montagem do laboratório descrito na seção 5.2 do roteiro e a execução do primeiro experimento (E1), com as evidências obtidas.

2. Pesquisa e questionário

A equipe redigiu uma primeira versão das respostas às questões comuns Q1 a Q14 e às questões específicas S1 a S4. Essas respostas serão revisadas e passarão a apontar para os experimentos que as sustentam (por exemplo, Q5 e o E3, Q6 e o E4, Q11 e o E6). Pontos já identificados para revisão:

- a comunicação entre servidores autoritativos segue o modelo primário–secundário, e não peer-to-peer;
- a resolução hierárquica (raiz, TLD e autoritativo) e o papel do cache em cada etapa;
- o serial do registro SOA como mecanismo de ordenação de versões, relacionado a relógios lógicos (RFC 1982);
- a posição do DNS diante de partições de rede, que será sustentada pelo resultado do E6.

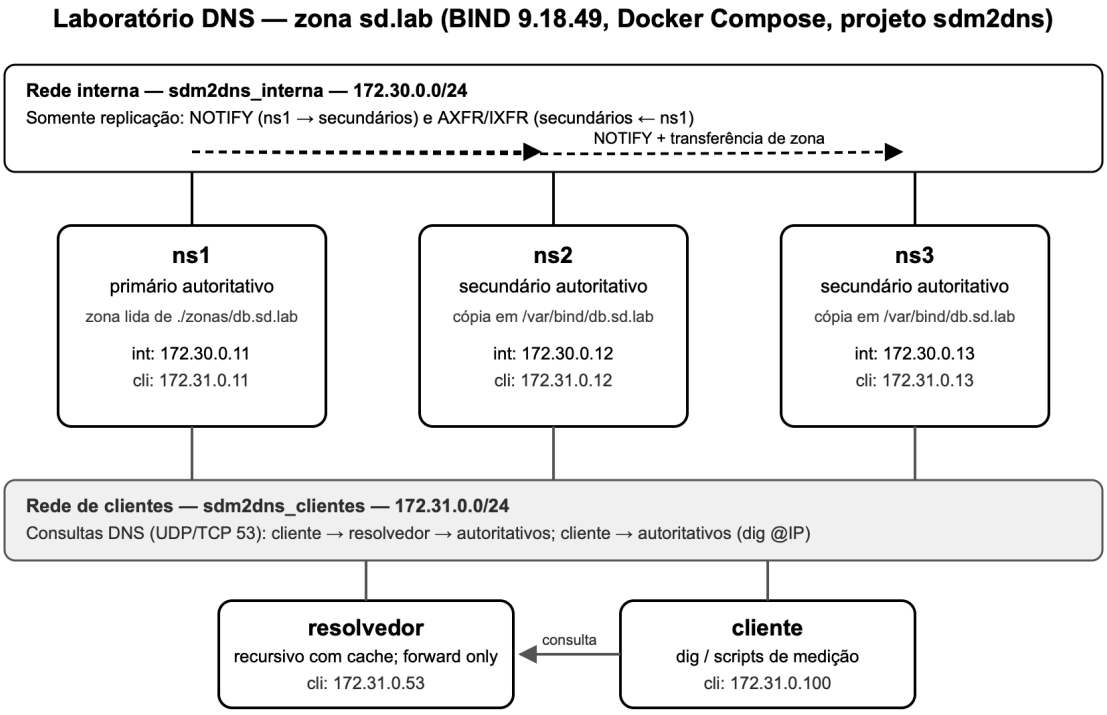
Fontes primárias em uso: Mockapetris e Dunlap (1988); RFC 1034, 1035, 1982, 1995, 1996, 2136 e 2308; manual de referência do BIND 9; documentação do CoreDNS.

3. Ambiente de testes

O laboratório roda inteiramente em contêineres leves com Docker Compose, sem máquinas virtuais adicionais, em um MacBook Air M3 (16 GB, macOS 26.4.1) com Docker Desktop 29.4.3 e Compose v5.1.3. Todos os nós usam a mesma imagem, construída pela equipe a partir do Alpine Linux 3.20 com o BIND 9.18.49 (39,1 MB). A topologia segue o roteiro: uma rede interna, usada só para a replicação da zona, e uma rede de clientes, usada para as consultas.

Nó	Função	Rede interna	Rede de clientes
ns1	Primário autoritativo da zona `sd.lab`	172.30.0.11	172.31.0.11
ns2	Secundário autoritativo	172.30.0.12	172.31.0.12
ns3	Secundário autoritativo	172.30.0.13	172.31.0.13
resolvedor	Recursivo com cache	—	172.31.0.53
cliente	Executa as consultas (`dig`)	—	172.31.0.100

Figura 1 — Topologia do laboratório DNS



Fonte: elaborado pelos autores (2026).

A zona `sd.lab` usa os valores do roteiro: serial 2026101401, refresh 30 s, retry 10 s, expire 300 s, TTL negativo 20 s e TTL padrão de 60 s. Os arquivos do roteiro foram completados e validados com `named-checkconf`, `named-checkzone` e `docker compose config`. Foram feitos três ajustes:

- geração da chave do `rndc` na imagem, sem a qual o comando `rndc reload` do roteiro não funciona;
- envio de NOTIFY apenas pela rede interna (`notify explicit`), para que toda a replicação passe por ela, condição necessária para o experimento de partição (E6);
- nome de projeto fixo (`sdm2dns`), para que as redes tenham sempre o mesmo nome.

Os arquivos de configuração, os scripts e as saídas brutas estão versionados em um repositório Git.

4. Experimento E1 — Subida e topologia

4.1. Previsão

Registrada antes da execução: os cinco contêineres sobem; os secundários, que começam sem cópia da zona, recebem do primário uma transferência completa (AXFR) pela rede interna; os três autoritativos passam a responder o mesmo serial, com a flag `aa` (resposta autoritativa); o resolvedor responde com a flag `ra` (recursão disponível) e sem `aa`.

4.2. Execução e evidências

Executado em 21/10/2026, às 19h34. Estado dos contêineres logo após a subida:

```
$ docker compose ps
NAME         SERVICE      STATUS
cliente      cliente      Up 5 seconds
ns1          ns1          Up 5 seconds
ns2          ns2          Up 5 seconds
ns3          ns3          Up 5 seconds
resolvedor   resolvedor   Up 5 seconds
```

Log do secundário ns2 (trecho; data e prefixo do contêiner omitidos): o NOTIFY do primário chega pelo endereço interno e dispara a transferência:

```
$ docker compose logs ns2 | grep -i transfer
22:34:55.411 client 172.30.0.11#39939: received notify for zone 'sd.lab'
22:34:55.412 zone sd.lab/IN: Transfer started.
22:34:55.412 transfer of 'sd.lab/IN' from 172.30.0.11#53: connected [...]
22:34:55.414 transfer of 'sd.lab/IN' from 172.30.0.11#53: Transfer completed:
             1 messages, 9 records, 224 bytes, 0.002 secs (serial 2026101401)
```

No primário, o mesmo evento aparece como `AXFR started` e `AXFR ended` para os clientes 172.30.0.12 (ns2) e 172.30.0.13 (ns3).

Consulta do registro SOA diretamente em cada servidor autoritativo:

```
$ for s in 11 12 13; do docker exec cliente dig @172.31.0.$s sd.lab SOA +short; done
ns1.sd.lab. admin.sd.lab. 2026101401 30 10 300 20
ns1.sd.lab. admin.sd.lab. 2026101401 30 10 300 20
ns1.sd.lab. admin.sd.lab. 2026101401 30 10 300 20
```

Mesma consulta a um nome, direto no autoritativo e pelo resolvedor:

```
$ docker exec cliente dig @172.31.0.11 app.sd.lab +norecurse
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 3, ADDITIONAL: 4
app.sd.lab.     60  IN  A  10.0.0.1
$ docker exec cliente dig @172.31.0.53 app.sd.lab
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
app.sd.lab.     60  IN  A  10.0.0.1
```

Previsão	**Resultado observado**
Cinco contêineres em execução	Confirmado, com os endereços da tabela da seção 3
Secundários recebem AXFR pela rede interna	Confirmado nos logs do primário e dos dois secundários (9 registros, 224 bytes)
Mesmo serial nos três autoritativos, com `aa`	Confirmado: 2026101401 nos três
Resolvedor com `ra` e sem `aa`	Confirmado

Uso de memória medido com `docker stats`: cerca de 26 MiB por servidor BIND, aproximadamente 104 MiB no total, compatível com os cerca de 100 MB indicados no roteiro.

4.3. Análise

O E1 confirma a arquitetura primário–secundário. O ns1 é o coordenador, definido de forma estática na configuração, e é o único que aceita alterações na zona. O ns2 e o ns3 são réplicas somente leitura que também respondem com autoridade. A replicação é iniciada pelos secundários: o NOTIFY apenas avisa que há uma versão nova (RFC 1996), e a cópia dos dados acontece na transferência de zona. A primeira sincronização foi completa

(AXFR), porque um secundário sem cópia anterior não tem a partir de que versão pedir uma transferência incremental (RFC 1995). O serial do SOA identifica a versão da zona, e o mesmo serial nos três servidores é a evidência de que convergiram.

Um detalhe não previsto: o ns2 transferiu a zona ao receber o NOTIFY, enquanto o ns3 transferiu cerca de 0,5 s depois pela sua própria verificação inicial, e o NOTIFY chegou a ele quando a zona já estava atualizada. A diferença entre os dois caminhos (NOTIFY e verificação periódica pelo refresh) será medida no E3.

5. Próximas etapas

- **E2 e E3:** caminho de uma leitura e de uma escrita; tempo de propagação do primário para os secundários com e sem NOTIFY, com cinco medições por cenário, média e desvio padrão.
- **E4:** cache, TTL e cache negativo no resolvedor; janela de leitura desatualizada.
- **E5 a E7:** falha de um e de dois servidores; partição do ns3 apenas na rede interna; recuperação.
- **E8:** serial do SOA como mecanismo de ordenação, incluindo a publicação de um serial menor.
- **E9 (ênfase D):** descoberta de serviços com CoreDNS, registro e remoção de instâncias, e efeito do TTL sobre a carga e sobre o tempo até o cliente deixar de usar uma instância removida.
- Revisão do questionário com base nas evidências, redação do relatório e preparação da apresentação de 28/10.